INTERNAL TECHNICAL DOCUMENT — BUILD SPECIFICATION

WhatsApp Bot & Conversation Viewer

Direct Meta Cloud API connection replacing Chatbase, with a FastAPI application and Supabase database providing message logging and a read-only conversation viewer. The trained n8n bot is used as-is via its API.

Version: 1.1 **Date:** 26 August 2026 **Timeline:** 4–5 days **Audience:** Development team

Stack: Python / FastAPI · Supabase · n8n · Render · GitHub

1 Scope & stack

Build a WhatsApp assistant that runs entirely on our own infrastructure. Chatbase is removed from the path. The FastAPI application is the single point of contact with Meta — it receives every inbound message, logs it, calls the **n8n API** for the reply, and sends that reply back through the Cloud API. The same application serves the password-protected conversation viewer.

> **The bot itself is already built and trained in n8n.** We are not rebuilding it, and we are not moving its prompt, model settings or knowledge sources into our database. n8n owns all of that. Our application's job is narrower and clearer: **connect to the n8n API, get the reply, deliver it to WhatsApp, and log everything.** Do not build a training or prompt-editing UI.

LAYER	TECHNOLOGY	NOTES
Application	Python 3.11 + FastAPI + Uvicorn	Jinja2 templates for the web pages; async throughout
Database	Supabase (Postgres)	Accessed over `DATABASE_URL` with asyncpg/SQLAlchemy; service-role key server-side only
Bot engine	n8n + OpenAI	**Already trained — used as-is via its API.** n8n owns the prompt, model settings and sources
Messaging	Meta WhatsApp Cloud API	Our own Meta app on a developer Facebook account, connected to our WABA
Hosting	Render (free plan)	Deployed from GitHub; see section 12 for the free-plan constraints
Source control	GitHub	Repo is the source of truth — must run locally from a clean clone

> **Build order.** Everything is built and proven **locally first**, driven entirely from what is in the GitHub repo. A teammate must be able to clone, follow the README, and have a working bot against the test number without asking anyone for missing pieces. Only once that is true do we deploy to Render.

2 Architecture & message flow

The application never makes the customer wait on OpenAI. Meta gets an immediate `200`; the reply is produced and sent asynchronously.

1. **Customer → Meta.** Customer sends a WhatsApp message to our business number.
2. **Meta → App.** Meta POSTs the message to `POST /webhook/whatsapp` on our FastAPI service.
3. **Verify & ack.** App validates the `X-Hub-Signature-256` HMAC, de-duplicates on `wa_message_id`, schedules background work and returns `200` immediately — target under 300 ms.

4. **Log inbound.** Background task upserts the contact, resolves or opens the conversation, and writes the inbound row to `messages`.
5. **App → n8n API.** App POSTs a normalised payload to the n8n webhook: message text, `wa_id`, profile name, `conversation_id`, `message_id`.
6. **n8n replies.** The existing trained workflow runs its agent and produces the reply text. Nothing about the bot's behaviour lives in our app.
7. **n8n → App.** n8n POSTs the reply to `POST /internal/reply` with the shared-secret header and the same `conversation_id`.
8. **App → Meta.** App calls the Cloud API `/{phone_number_id}/messages` endpoint, then writes the outbound row to `messages` with the returned Meta message ID.
9. **Viewer.** Both rows are already in Supabase, so the conversation is visible in the web app within a second of it happening.

> **Why the callback and not a synchronous call.** Do not hold the request open waiting for n8n. OpenAI latency is unbounded, Meta retries webhooks it thinks failed, and a blocked worker under load produces duplicate replies to real customers. Fire to n8n, return, let n8n call us back.

3 Repository structure

```
whatsapp-bot/
├── app/
│   ├── main.py                # FastAPI app, router registration, startup checks
│   ├── config.py              # Pydantic Settings — all env vars, fails fast if missing
│   ├── deps.py                # DB session, auth dependency, shared-secret guard
│   ├── routers/
│   │   ├── webhook.py         # GET + POST /webhook/whatsapp
│   │   ├── internal.py        # POST /internal/reply  (n8n callback)
│   │   ├── auth.py            # login / logout / session
│   │   ├── viewer.py          # conversations, thread, leads, search, error log
│   │   └── export.py          # CSV / XLSX download
│   ├── services/
│   │   ├── meta_client.py     # send message, signature verify, media handling
│   │   ├── n8n_client.py      # call the n8n API, retry + backoff
│   │   ├── conversation.py    # contact upsert, 24h window
│   │   ├── message_log.py     # inbound/outbound persistence
│   │   └── errors.py          # error_log writer + fallback reply sender
│   ├── db/
│   │   ├── session.py         # async engine / pool
│   │   ├── models.py          # SQLAlchemy models
│   │   └── queries.py         # viewer queries: list, thread, search, leads
│   ├── templates/             # base, login, conversations, thread, leads
│   └── static/                # css, minimal js — no CDN dependencies
├── migrations/
│   └── 001_init.sql           # run in Supabase SQL editor or via psql
├── tests/
│   ├── test_webhook.py        # signature, dedup, fast-ack
│   ├── test_conversation.py   # windowing, history
│   └── test_viewer.py         # search, filters, export
├── .env.example               # every var, no real values — committed
├── .gitignore                 # must include .env
├── requirements.txt
├── render.yaml                # Render blueprint
└── README.md                  # local setup, start to finish
```

Never commit secrets. `.env` is gitignored. `.env.example` lists every variable with placeholder values. Any key that reaches a commit is treated as compromised and rotated immediately — Meta tokens and the Supabase service-role key especially.

4 Database schema (Supabase)

Run `migrations/001_init.sql` in the Supabase SQL editor. Timestamps are stored in UTC (`timestamptz`) and rendered in local time by the app.

```
create table contacts (
  id             bigserial primary key,
  wa_id          text unique not null,        -- E.164, no '+'
  profile_name   text,
  first_seen_at  timestamptz not null default now(),
  last_seen_at   timestamptz not null default now(),
  message_count  integer not null default 0
);

create table conversations (
  id              bigserial primary key,
  contact_id      bigint not null references contacts(id) on delete cascade,
  started_at      timestamptz not null default now(),
  last_message_at timestamptz not null default now(),
  message_count   integer not null default 0
);
```

```
create table messages (
  id              bigserial primary key,
  conversation_id bigint not null references conversations(id) on delete cascade,
  contact_id      bigint not null references contacts(id) on delete cascade,
  wa_message_id   text unique,                -- Meta id; drives de-duplication
  direction       text not null check (direction in ('customer','bot')),
  body            text,
  msg_type        text not null default 'text',
  media_url       text,
  sent_at         timestamptz not null,       -- Meta timestamp for inbound
  meta_status     text,                       -- sent / delivered / read / failed
  created_at      timestamptz not null default now()
);

create table error_log (
  id              bigserial primary key,
  conversation_id bigint references conversations(id) on delete set null,
  wa_id           text,
  inbound_body    text,
  step            text not null,              -- webhook / n8n / openai / meta_send / db
  error_text      text not null,
  payload         jsonb,
  created_at      timestamptz not null default now()
);

-- NOTE: there is no bot_config or knowledge_sources table.
-- The bot is trained in n8n and n8n owns its prompt, model and sources.

create table app_users (
  id            bigserial primary key,
  email         text unique not null,
  password_hash text not null,                -- argon2 or bcrypt
  role          text not null default 'viewer',
  created_at    timestamptz not null default now(),
  last_login_at timestamptz
);

-- indexes that matter
create index on messages (conversation_id, sent_at desc);
create index on messages (contact_id, sent_at desc);
create index on conversations (last_message_at desc);
create index on messages using gin (to_tsvector('english', coalesce(body,'')));
create index on error_log (created_at desc);
```

Conversation windowing

On each inbound message, find the contact's most recent conversation. If its `last_message_at` is **within 24 hours**, append to it. Otherwise open a new conversation row. This keeps threads readable in the viewer and gives n8n a stable `conversation_id` to key its own memory on. Keep the 24-hour constant in `config.py`, not scattered through the code.

5 Environment variables

Defined once in `app/config.py` as a Pydantic `Settings` class. The app must refuse to start if any required variable is missing — a half-configured service that boots is worse than one that fails loudly.

VARIABLE	PURPOSE
`META_APP_ID`	Meta app ID from the developer dashboard
`META_APP_SECRET`	Used to compute the `X-Hub-Signature-256` HMAC
`META_VERIFY_TOKEN`	Our own random string, echoed during webhook verification
`META_ACCESS_TOKEN`	Permanent system-user token (not the 24-hour temp token)
`META_PHONE_NUMBER_ID`	Sending number's ID — not the phone number itself
`META_WABA_ID`	WhatsApp Business Account ID
`META_API_VERSION`	Graph API version, e.g. `v21.0` — pinned, never left floating
`DATABASE_URL`	Supabase Postgres connection string (pooler URI)
`SUPABASE_URL` / `SUPABASE_SERVICE_ROLE_KEY`	Only if the Supabase client SDK is used; server-side only, never sent to the browser
`N8N_WEBHOOK_URL`	The trained bot's n8n API endpoint — where the app dispatches inbound messages
`N8N_CALLBACK_SECRET`	Shared secret n8n sends back on `/internal/reply`
`SESSION_SECRET`	Signs viewer session cookies
`APP_BASE_URL`	Public URL — tunnel URL locally, Render URL in production
`FALLBACK_MESSAGE`	Text sent to the customer when the workflow fails
`LOG_LEVEL` / `TZ`	`INFO` in production; `Asia/Karachi` for display

6 Meta app setup (developer Facebook account)

1. At `developers.facebook.com`, create a new app — type **Business** — under the developer Facebook account.
2. Add the **WhatsApp** product. This provisions a test number and a sandbox WABA for development.
3. Note the **Phone number ID** and **WABA ID** from the API Setup panel.
4. The temporary token shown there expires in 24 hours — fine for the first local test, useless after that.
5. For a lasting token: in Business Settings create a **System User**, assign the WABA as an asset, and generate a token with `whatsapp_business_messaging` and `whatsapp_business_management`. That is `META_ACCESS_TOKEN`.
6. Under WhatsApp → Configuration, set the **Callback URL** to `{APP_BASE_URL}/webhook/whatsapp` and the **Verify token** to `META_VERIFY_TOKEN`. Meta immediately sends a `GET` — the app must be running and reachable at that moment.
7. Subscribe the webhook to the **messages** field. Without this, verification succeeds and no messages ever arrive — a common half-hour lost.
8. Add each developer's personal WhatsApp number as a recipient in the test-number panel; the sandbox will only message approved numbers.

> **Production number.** The live number is registered in our own WABA later, per the agreed plan. The old number is attached to the same WABA afterwards so both route to the same bot. All of that changes only `META_PHONE_NUMBER_ID` — the application code is number-agnostic. Build it that way; do not hardcode a number anywhere.

7 API endpoints

METHOD	PATH	BEHAVIOUR
GET	`/webhook/whatsapp`	Meta verification. Compare `hub.verify_token`; return `hub.challenge` as plain text, else `403`.
POST	`/webhook/whatsapp`	Verify HMAC → parse → dedup on `wa_message_id` → schedule background task → return `200` immediately.
POST	`/internal/reply`	n8n API callback. Guarded by `N8N_CALLBACK_SECRET`. Body: `{conversation_id, wa_id, text}`. Sends via Cloud API, logs outbound.
GET	`/health`	Returns `200` plus a DB ping. Used by Render health checks and the keep-alive pinger.
GET/POST	`/login`, `/logout`	Session login against `app_users`. Everything below requires a valid session.
GET	`/conversations`	List newest first — number, name, last message time, message count. Paginated, 50 per page.
GET	`/conversations/{id}`	Full thread, WhatsApp-style: customer left, bot right, timestamps.
GET	`/search`	Query by phone number or keyword (full-text on `body`), with `from` / `to` date filters.
GET	`/leads`	Unique contacts: number, name, first contact, last contact, message count.
GET	`/export/leads.csv` · `.xlsx`	Same result set as the current filter, streamed as a download.
GET	`/errors`	Recent rows from `error_log` — step, message, time. For support during the warranty month.

> **The application is read-only in both directions.** There is no send-message endpoint for humans and no reply box in the UI; outbound messages originate only from `/internal/reply`. There is also **no prompt or model editing** — the bot is configured in n8n. Both are explicit scope decisions, not oversights. Do not add either.

8 n8n API contract

The bot already exists and is trained in n8n. Our application connects to its API and does nothing more than that. Two fixed payloads — agree them with whoever owns the workflow before either side is built, because changing them later means changing both.

App → n8n (`POST {N8N_WEBHOOK_URL}`)

```
{
  "conversation_id": 4812,
  "wa_id": "923001234567",
  "profile_name": "Ahmed",
  "message": "Do you deliver to Lahore?",
  "message_id": "wamid.HBgM...",
  "callback_url": "https://<app>/internal/reply"
}

// No prompt, no model settings, no knowledge sources in this payload.
// n8n owns all of that. conversation_id is the stable key n8n uses
// for its own conversation memory.
```

n8n → App (`POST /internal/reply`)

```
Header: X-Callback-Secret: <N8N_CALLBACK_SECRET>

{
  "conversation_id": 4812,
  "wa_id": "923001234567",
  "text": "Yes — we deliver across Lahore within 2 working days.",
  "status": "ok"          // "error" makes the app send FALLBACK_MESSAGE instead
}
```

> **Confirm the n8n side before day 2.** We need three things from whoever owns the workflow: the production webhook URL, whether it accepts our field names or needs a mapping node, and whether it keys memory on `conversation_id` or on `wa_id`. Getting this wrong is the most likely cause of a lost day — settle it in writing at the start, not by trial and error mid-build.

9 Error handling — no silent failures

Every path that can fail writes to `error_log` and, where a customer is waiting, sends the fallback reply. A customer must never be left with silence.

FAILURE	HANDLING
Bad or missing HMAC signature	Log at warn, return `403`. Do not process. Do not reply.
Duplicate `wa_message_id`	Return `200`, do nothing else. Meta retries are normal and must be idempotent.
n8n unreachable / non-2xx	Retry 3× with exponential backoff (2s, 4s, 8s). On final failure: write `error_log`, send `FALLBACK_MESSAGE`.
No callback within 60s	Timeout watchdog fires: log the timeout, send the fallback so the customer is not stranded.
n8n returns `status:"error"`	Log the reported step and error, send the fallback.
Meta send fails	Retry twice, then log with the full Meta error body. Meta error codes are specific — store the code, not just the message.
Database unreachable	Still attempt the reply path; log to stdout so Render captures it. Never let a logging failure block a customer reply.

`FALLBACK_MESSAGE` should read as a person, not a stack trace — for example: *"Sorry, I'm having trouble answering right now. Please try again in a moment."* Every fallback sent is logged as an outbound message so it appears in the viewer thread.

10 Local development setup

This is the sequence the README must document. Target: clean clone to working bot in under 30 minutes.

```
# 1. clone and isolate
git clone git@github.com:<org>/whatsapp-bot.git && cd whatsapp-bot
python3.11 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# 2. configure
cp .env.example .env        # fill in Meta + Supabase values

# 3. schema — paste migrations/001_init.sql into the Supabase SQL editor,
#    or: psql "$DATABASE_URL" -f migrations/001_init.sql

# 4. seed the first admin user and default bot config
python -m app.scripts.seed_admin --email you@company.com

# 5. run
uvicorn app.main:app --reload --port 8000

# 6. expose localhost so Meta can reach it
ngrok http 8000             # or: cloudflared tunnel --url http://localhost:8000
#    put the https URL in .env as APP_BASE_URL, and in the Meta dashboard
#    as the Callback URL: https://<tunnel>/webhook/whatsapp
```

1. Save the webhook in the Meta dashboard — verification must return your challenge and go green.
2. Subscribe to the `messages` field if not already done.
3. Message the test number from an approved WhatsApp number.
4. Confirm: a row in `messages` with `direction='customer'`, a dispatch to n8n, a callback, an outbound row, and the reply on your phone.
5. Open `http://localhost:8000/conversations` and confirm the thread renders.

> **The tunnel URL changes every restart** on free ngrok. Each time it changes, update both `APP_BASE_URL` and the Meta callback URL, or messages will silently stop arriving. This catches everyone at least once — check it before debugging anything else.

11 Web application

Viewer (login required)

- **Conversations** — newest first: number, WhatsApp name, last message time, message count. Paginated.
- **Thread** — full transcript, WhatsApp-style bubbles, customer left, bot right, timestamps, day separators.
- **Search** — phone number or keyword across message bodies, using the GIN full-text index.
- **Date filter** — single day or range, combinable with search.
- **Leads** — unique contacts with name, first contact, last contact, message count.
- **Export** — current filtered result set to CSV and XLSX.

Operations

- **Users** — create and disable viewer logins. Seeded via script; a simple page is enough.
- **Error log** — recent failures with step, message and time, for support during the warranty month.
- **No bot training UI.** Prompt, model settings and knowledge sources live in n8n and are edited there. The app does not read them, store them or display them.

> Dropping the training panel is what makes the 4–5 day timeline achievable. If someone asks for prompt editing in the web app later, it is new scope — quote it, do not absorb it.

Server-rendered Jinja2, plain CSS, no build step and no CDN dependencies. This is an internal tool for a handful of users — a JavaScript framework here buys nothing and costs deployment complexity.

12 Deployment to Render (free plan)

1. Push to GitHub. Connect the repo in Render → **New Web Service**.
2. Runtime Python 3.11. Build: `pip install -r requirements.txt`. Start: `uvicorn app.main:app --host 0.0.0.0 --port $PORT`.
3. Add every variable from `.env.example` in the Render dashboard. Set `APP_BASE_URL` to the Render URL.
4. Set the health check path to `/health`.
5. Update the Meta callback URL to `https://<service>.onrender.com/webhook/whatsapp` and re-verify.
6. Send a live test message and confirm the full round trip before considering it deployed.

> **Free-plan constraint you must design around.** Render spins a free web service down after **15 minutes without inbound traffic**, and spin-up takes roughly a minute. Because our app — not n8n — receives the Meta webhook, a sleeping service means the first message after a quiet period is delayed by up to a minute or dropped while Meta retries. Mitigations, in order:
>
> - **Keep-alive ping** — an external cron (cron-job.org, UptimeRobot) hitting `/health` every 10 minutes. One always-on service fits inside the 750 free instance-hours per month, but it consumes essentially all of them, so keep this to a single free service in the workspace.
> - **Idempotency is not optional** — Meta retries on timeout, so the `wa_message_id` dedup in section 9 is what prevents a cold start turning into duplicate replies.
> - **Recommend the paid instance for production.** The free plan is right for building and demoing; a customer-facing webhook on a service that sleeps is a known liability. Raise this with the client before cutover rather than after the first missed message.

> **Do not use Render's free Postgres.** It expires 30 days after creation. Our database is Supabase — Render hosts the application only.

13 Security requirements

- **Webhook signature verification is mandatory.** Compute HMAC-SHA256 over the raw request body with `META_APP_SECRET` and compare to `X-Hub-Signature-256` using a constant-time comparison. Without it, anyone who finds the URL can inject fake customer messages.
- Read the **raw body** for the HMAC, before any JSON parsing — re-serialised JSON will not match.
- `/internal/reply` is guarded by `N8N_CALLBACK_SECRET`, also compared in constant time.

- Passwords hashed with **argon2** or **bcrypt**. Never plaintext, never a shared password in the code.
- Session cookies: `HttpOnly`, `Secure`, `SameSite=Lax`.
- Rate-limit `/login` to blunt brute force.
- Supabase service-role key stays server-side. If any table is exposed through the Supabase API, enable RLS.
- All secrets live in environment variables, in Render's dashboard and local `.env` only.
- Conversation data is customer PII. No production dumps on laptops, no exports in Slack or email.

14 Testing & acceptance checklist

Part A — messaging

- ☐ Webhook verification (GET) returns the challenge
- ☐ Valid signature accepted, invalid rejected with 403
- ☐ Webhook acks in under 300 ms under load
- ☐ Duplicate `wa_message_id` processed exactly once
- ☐ Inbound message logged with correct contact and conversation
- ☐ n8n API receives the payload and returns a reply
- ☐ Reply reaches the customer's phone
- ☐ Outbound logged with Meta message ID
- ☐ New conversation opens after a 24-hour gap
- ☐ n8n stopped → fallback sent, error logged
- ☐ Invalid Meta token → error logged with Meta's code
- ☐ Non-text message (image, voice) recorded by type, no crash

Part B — viewer

- ☐ Unauthenticated access redirects to login
- ☐ Conversations list ordered newest first
- ☐ Counts and last-message times are correct
- ☐ Thread renders in order with correct sides
- ☐ Search by phone number returns the right thread
- ☐ Keyword search matches message bodies
- ☐ Date filter works alone and with search
- ☐ Leads view shows unique contacts, no duplicates
- ☐ CSV and XLSX export open cleanly in Excel
- ☐ New message visible in the viewer **within 60 seconds**
- ☐ Error log page lists a forced failure
- ☐ Timestamps display in Asia/Karachi

15 Work breakdown — 4 to 5 days

Twelve tasks across five working days. Because the bot is already trained in n8n, there is no agent rebuild and no training UI — the app is a messaging bridge plus a viewer, and that is what the schedule below reflects. Day 5 is the buffer: if Meta and n8n cooperate, this finishes on day 4.

DAY	TASK	INCLUDES	PART
1	1. Repo & skeleton	Structure, FastAPI app, Pydantic settings, `/health`, README, `.env.example`, `.gitignore`	A
	2. Supabase schema	`001_init.sql`, indexes, seed-admin script, connection verified from local	A + B
	3. Meta app setup	Dev app, WhatsApp product, system-user token, webhook subscribed, tunnel verified	A
2	4. Webhook receiver	GET verify, POST fast-ack, HMAC signature check, dedup, background task	A
	5. Logging & conversation	Contact upsert, 24-hour windowing, inbound + outbound persistence	A + B
3	6. n8n API integration	Dispatch to the trained workflow, `/internal/reply` callback, shared-secret guard, retries	A
	7. Meta send & error handling	Cloud API client, retries, fallback message, `error_log`, 60s watchdog	A
4	8. Auth & viewer core	Login, sessions, conversations list, thread view	B
	9. Search, leads & export	Keyword + number search, date filter, leads view, CSV + XLSX export, error log page	B
5	10. End-to-end test	Full section 14 checklist against the test number, locally	A + B
	11. Render deployment	Service, env vars, health check, keep-alive ping, webhook re-point, live test	A + B

12. Migration & handover	Chatbase export/import, cutover, documentation, credential handover	A

Tasks 1, 3, 4, 6, 7 and 12 constitute **Part A**; tasks 2, 5, 8 and 9 constitute **Part B**. Part A is deployable and demonstrable without the viewer — build to that boundary so it can be signed off on its own.

16 Known risks

RISK	MITIGATION
Render free-tier cold starts delay live messages	Keep-alive ping + strict idempotency; recommend the paid instance before cutover
Supabase free projects pause after ~1 week idle	Not a concern on a live number, but never let the staging project sit idle before a demo
Meta business verification or number registration delays	Start the Meta work on day 1, in parallel with development — it is the longest external dependency
Chatbase export format unknown until we have it	Pull the export on day 1, not the day before cutover
n8n API contract not settled before day 2	Blocks task 6 and puts the whole 4–5 days at risk. Confirm URL, field names and memory key in writing on day 1
n8n workflow latency exceeds the watchdog	Fallback fires and is logged; tune the watchdog once real latency is measured
Graph API version deprecation	`META_API_VERSION` is pinned and reviewed, not left to default